SALESTORM | SYSCRAFTERS 2026

SYSTEM DESIGN HACKATHON

Team Name : Byte Blenderz

Team Members: VISWAJOTHI R [24AM119]
SAHANAA K [24AM084]
VISHVA KUMAR B[24A119]

Department : CSE(AI&ML)

Date : 05/10/2026

Requirements & Assumptions Document:

1.Business Problem:

SALESTORM is a high-scale e-commerce flash-sale scenario. During a limited-stock sale, a sudden surge of customers may place purchase requests at the same time. The core business problem is to handle thousands of simultaneous purchase requests for limited inventory without overselling, while keeping payment processing and the order lifecycle reliable when services experience delays or failures. The critical scenario is 10,000 concurrent purchase requests competing for only 100 available units. The required business flow is: Customer → Product Discovery → Cart → Inventory Check → Inventory Reservation → Checkout → Payment → Order → Fulfilment → Shipment → Notification → Delivery Tracking

2. Business Objectives:

- Support sudden increases in customer traffic without system collapse.
- Prevent inventory overselling under concurrent purchase requests.
- Provide temporary inventory reservation with expiry and release.
- Ensure safe and idempotent payment processing.
- Maintain a consistent and traceable order lifecycle.
- Recover gracefully from service, database and payment failures.
- Produce a design that an engineering team can understand, implement and operate

3.Functional Requirements:

ID	Functional requirement
FR-01	The system shall support the required customer purchase flow from product discovery through delivery tracking.
FR-02	The system shall check inventory before completing a purchase.
FR-03	The system shall support temporary inventory reservations.

FR-04	The system shall support reservation expiry and release.
FR-05	The system shall prevent duplicate reservations for repeated purchase requests.
FR-06	The system shall process payments safely and idempotently.
FR-07	The system shall handle payment success, payment failure and payment timeout.
FR-08	The system shall prevent duplicate payment transactions for duplicate payment requests.
FR-09	The system shall maintain a correct order lifecycle for successful purchases.
FR-10	The system shall provide recovery behaviour when the Order Service is temporarily unavailable.
FR-11	The system shall provide defined recovery paths for database and payment-gateway failures.
FR-12	The system shall support horizontal scaling for peak traffic.
FR-13	The system shall provide authentication, authorization, secure communication, input validation, payment-information protection, secrets management, audit logging, rate limiting and abuse protection.
FR-14	The system shall provide observability for critical transactions through metrics, structured logs, distributed tracing and alerts.

4. Non-Functional Requirements:

4.1 Throughput

• Normal traffic: approximately 10,000 requests/sec.

• Flash-sale traffic: the architecture should reason about up to 500,000 requests/sec.

• The design must explain how horizontal scaling, load balancing, caching, queueing and database scaling address the traffic envelope.

• These are traffic/design requirements from the brief; they are not claims of measured implementation performance.

4.2 Latency

The official brief identifies latency as a measurable non-functional requirement, but it does not specify a numeric latency target or SLA. Therefore, no numeric latency value is asserted as an official requirement in Stage 1. A specific latency target would be a later engineering target, not a source-of-truth requirement.

4.3 Availability

The official brief requires reliable workflows and graceful recovery from service, database and payment failures, but it does not specify a numeric availability percentage or SLA. Therefore, no numeric availability value is asserted as an official requirement in Stage 1.

4.4 Consistency

• 100 available units must not result in more than 100 successful sales. • Inventory must not become negative.

• Duplicate reservations must not create duplicate business reservations.

• Duplicate payment requests must not create duplicate payment transactions. • Reservation expiry and release must produce correct business outcomes.

• The order lifecycle must remain valid and traceable.

• Required reservation lifecycle: AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD.

• Required reservation failure paths: RESERVED → PAYMENT_FAILED → RELEASED; RESERVED → TIMEOUT → RELEASED. • Required order lifecycle: CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED.

4.5 Security
• Authentication and authorization.
• HTTPS / secure API communication.
• Rate limiting and abuse protection.
• Input validation.
• Secure handling of payment information.
• Secrets management.
• Audit logging.

4.6 Recovery
• Recover correctly from payment failure.
• Handle payment timeout using safe recovery behaviour.
• Prevent duplicate payment during retries.
• Release reservations after payment failure or timeout/expiry.
• Recover when the Order Service is unavailable after payment success.
• Provide recovery paths for database failure.
• Provide recovery paths for payment-gateway failure.
• Support retry, circuit-breaker, dead-letter handling, idempotent processing, and compensation or reconciliation where appropriate.

5.Assumption:

• The official brief does not specify a numeric latency target; no numeric latency SLA is assumed here.
• The official brief does not specify a numeric availability target; no numeric availability SLA is assumed here.
• The brief does not mandate a specific cloud provider, database, cache, message broker, programming language or framework.
• The brief permits alternative architectures when their trade-offs are justified; Stage 1 therefore records requirements without selecting an architecture or technology.
• Any later implementation-level values or technology choices must be treated as engineering decisions rather than official Stage 1 requirements.

6.Constraints:

• The challenge is design-first; production implementation is not required.
• The primary scenario is 10,000 simultaneous purchase requests competing for 100

available units.

• The design must prevent overselling and duplicate reservations/orders.

• Payment and order workflows must remain reliable under failures.

• The design must reason about approximately 10,000 requests/sec under normal traffic and up to 500,000 requests/sec during flash-sale traffic.

• AI-assisted coding may accelerate optional prototypes, tests, simulations, API skeletons and documentation, but must not replace requirement analysis, architecture selection, concurrency reasoning, inventory consistency decisions, payment/order state modelling, trade-off analysis, security/reliability decisions or the final technical explanation.

7.Expected Traffic

Scenario	Requirement	Classification
Normal traffic	≈ 10,000 requests/sec	Official requirement
Flash-sale traffic	Reason about up to 500,000 requests/sec	Official design requirement
Concurrent purchase scenario	10,000 simultaneous purchase requests	Official challenge scenario
Available stock	100 units	Official challenge scenario

8. Critical Dependencies:

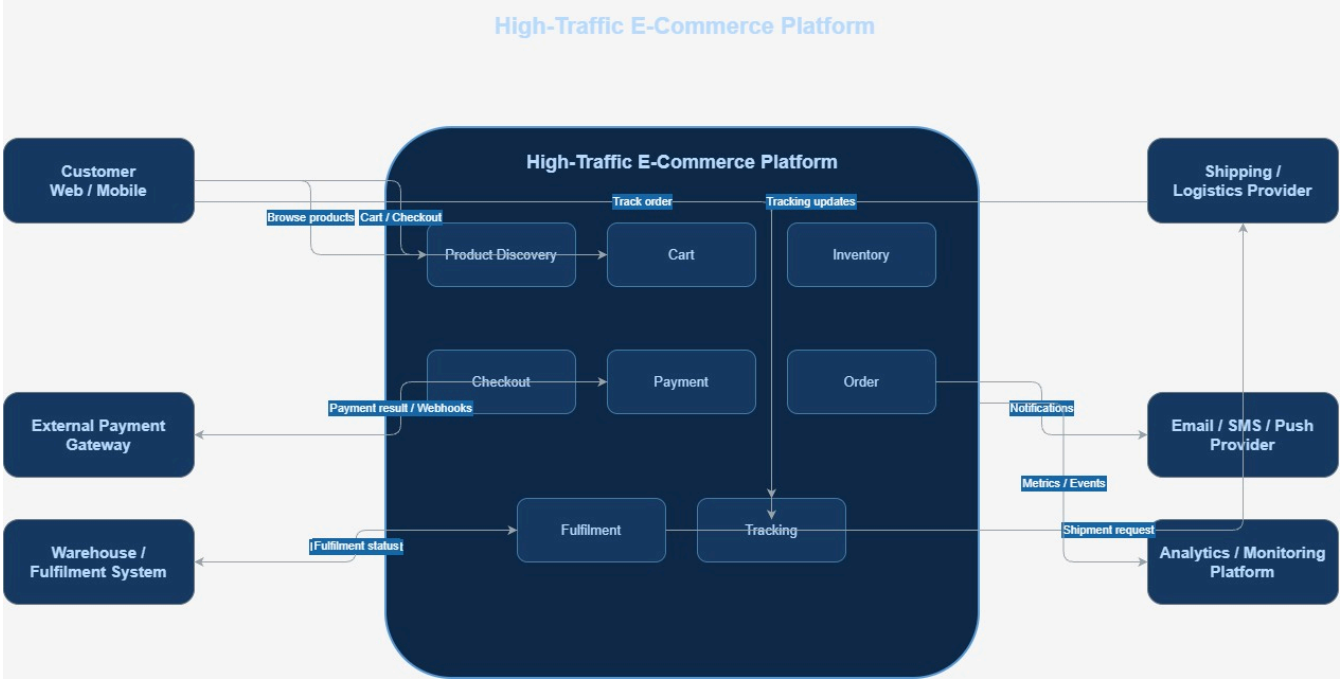
• Payment gateway / payment system.
• Database.
• Order Service.
• Fulfilment and shipment systems.
• Notification systems.
• External systems.
• Caching and message-queue infrastructure were used by the final architecture.
• Monitoring, logging and distributed-tracing infrastructure. These are dependency categories identified or required by the official brief. The brief does not mandate specific vendors or products.

9.Strict Guarantees vs Targets:

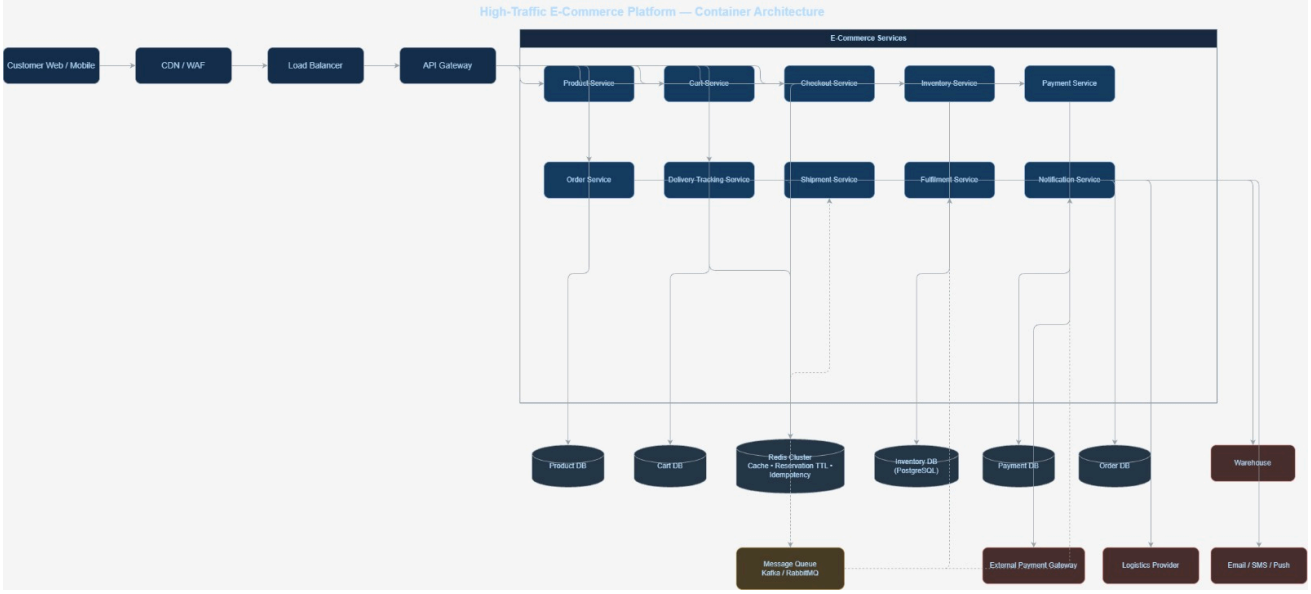
Requirement	Classification	Meaning
Inventory overselling	STRICT GUARANTEE	100 available units cannot result in more than 100 successful sales.
Inventory quantity correctness	STRICT GUARANTEE	Successful reservations/sales must not cause inventory to become negative.
Duplicate payment	STRICT GUARANTEE	Duplicate payment requests must not create duplicate transactions.
Reservation expiry/release	STRICT GUARANTEE	Unpaid or expired reservations are released correctly.
Order lifecycle	STRICT GUARANTEE	Successful purchases eventually reach a valid order state and lifecycle transitions remain correct.
Reservation duplicate prevention	STRICT GUARANTEE	Duplicate reservation requests must not create duplicate reservations.
≈10,000 requests/sec normal traffic	TARGET / TRAFFIC ENVELOPE	Official normal-traffic design requirement.
Up to 500,000 requests/sec flash-sale traffic	TARGET / DESIGN ENVELOPE	Official flash-sale traffic envelope to reason about.
10,000 simultaneous purchase requests	REQUIRED SCENARIO	Official concurrency challenge.
Recovery from service/payment/database failures	REQUIRED RELIABILITY OUTCOME	Defined recovery paths are required; no numeric recovery SLA is specified.
Latency	NOT NUMERICALLY SPECIFIED	Brief requires measurable latency consideration but provides no numeric SLA.
Availability	NOT NUMERICALLY SPECIFIED	Brief requires reliability/recovery but provides no numeric uptime SLA.

High Level Design:

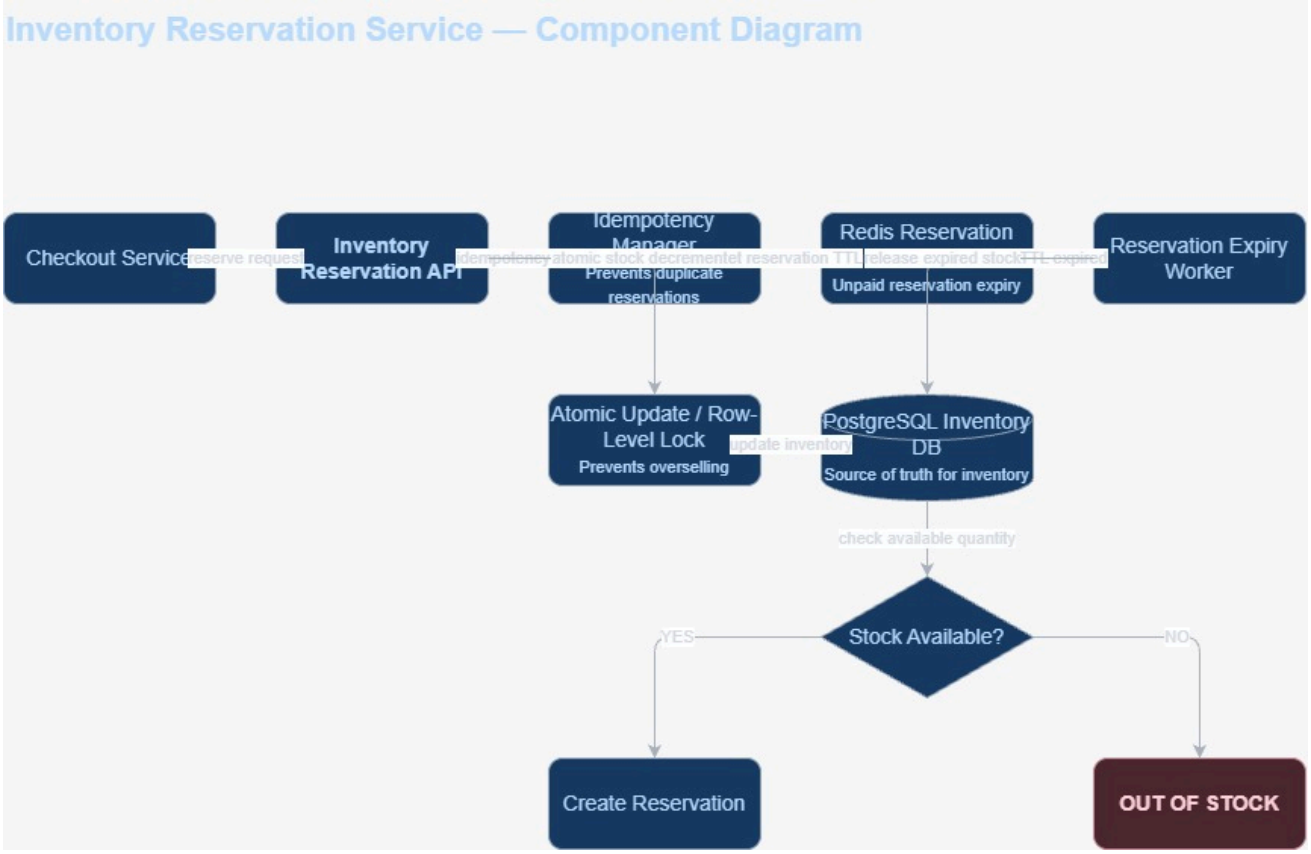
System Context Diagram:



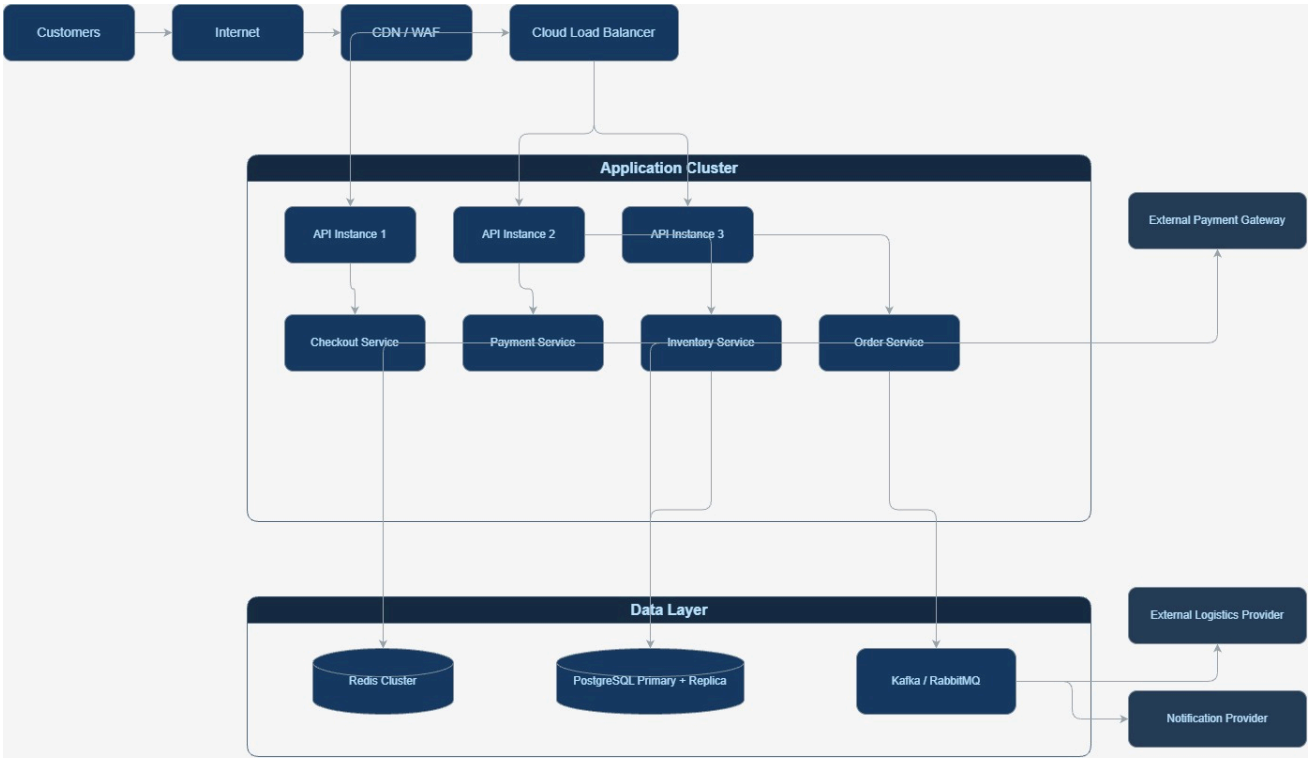
Container Diagram:



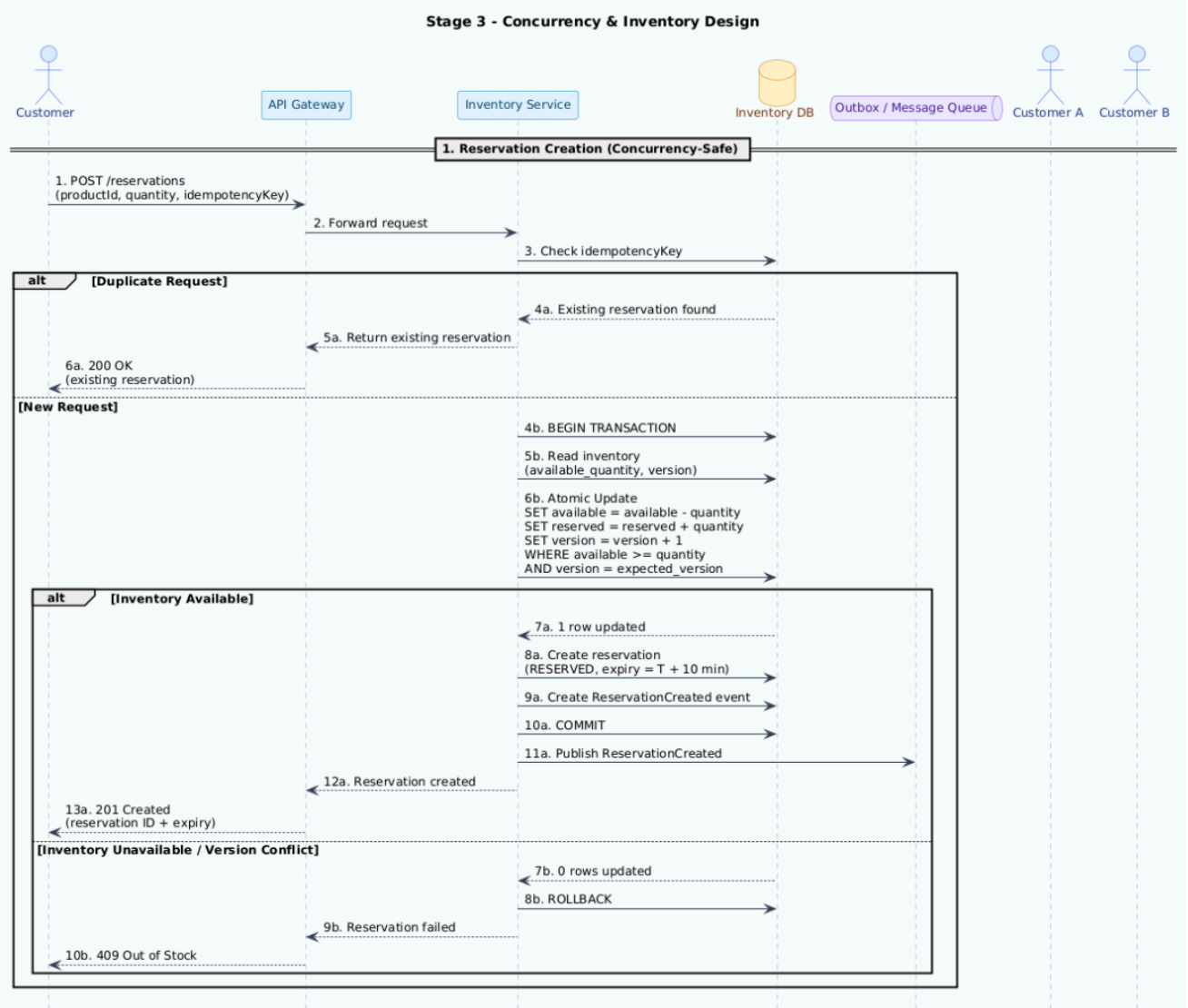
Component Diagram:

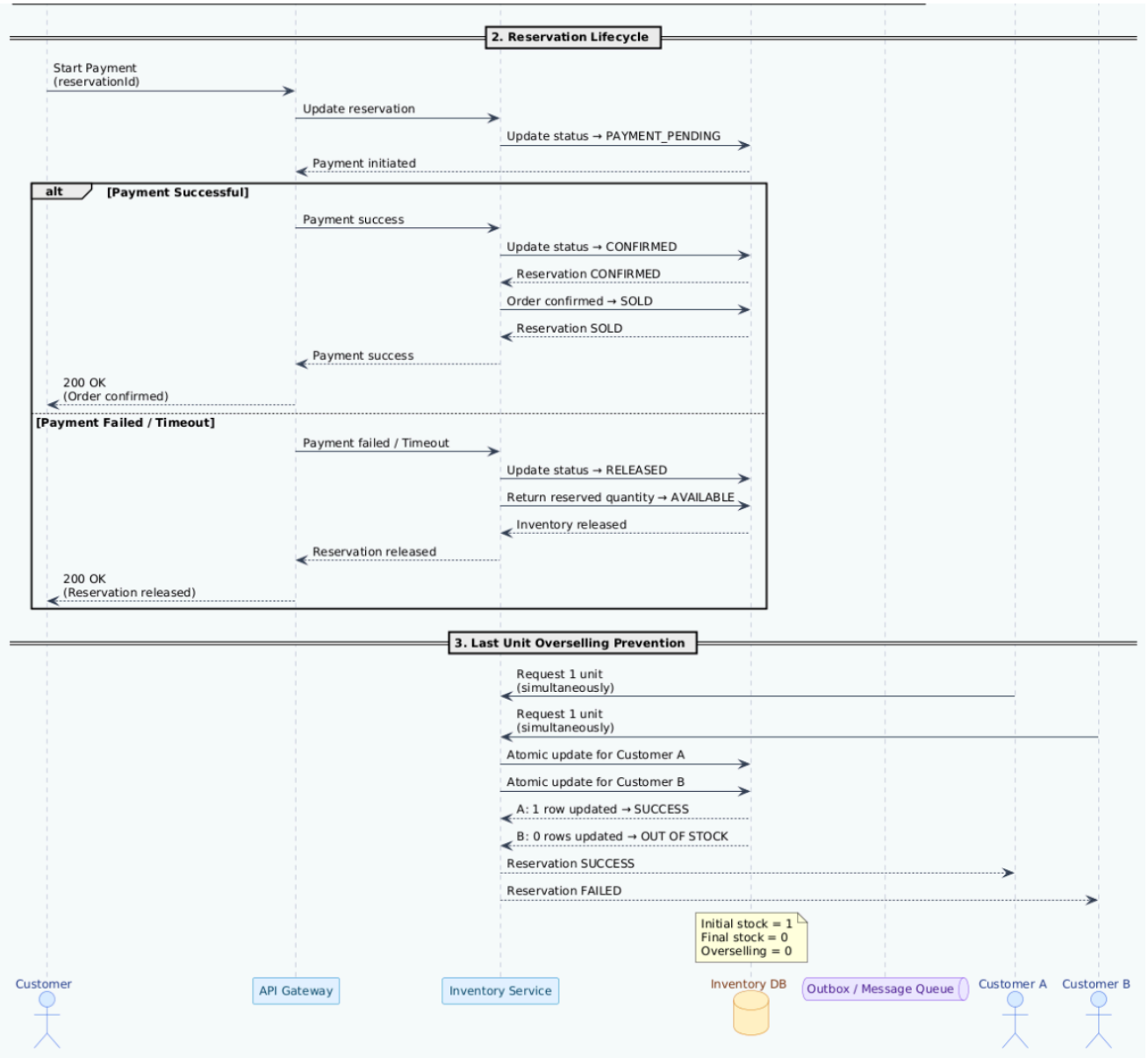


Deployment Diagram:



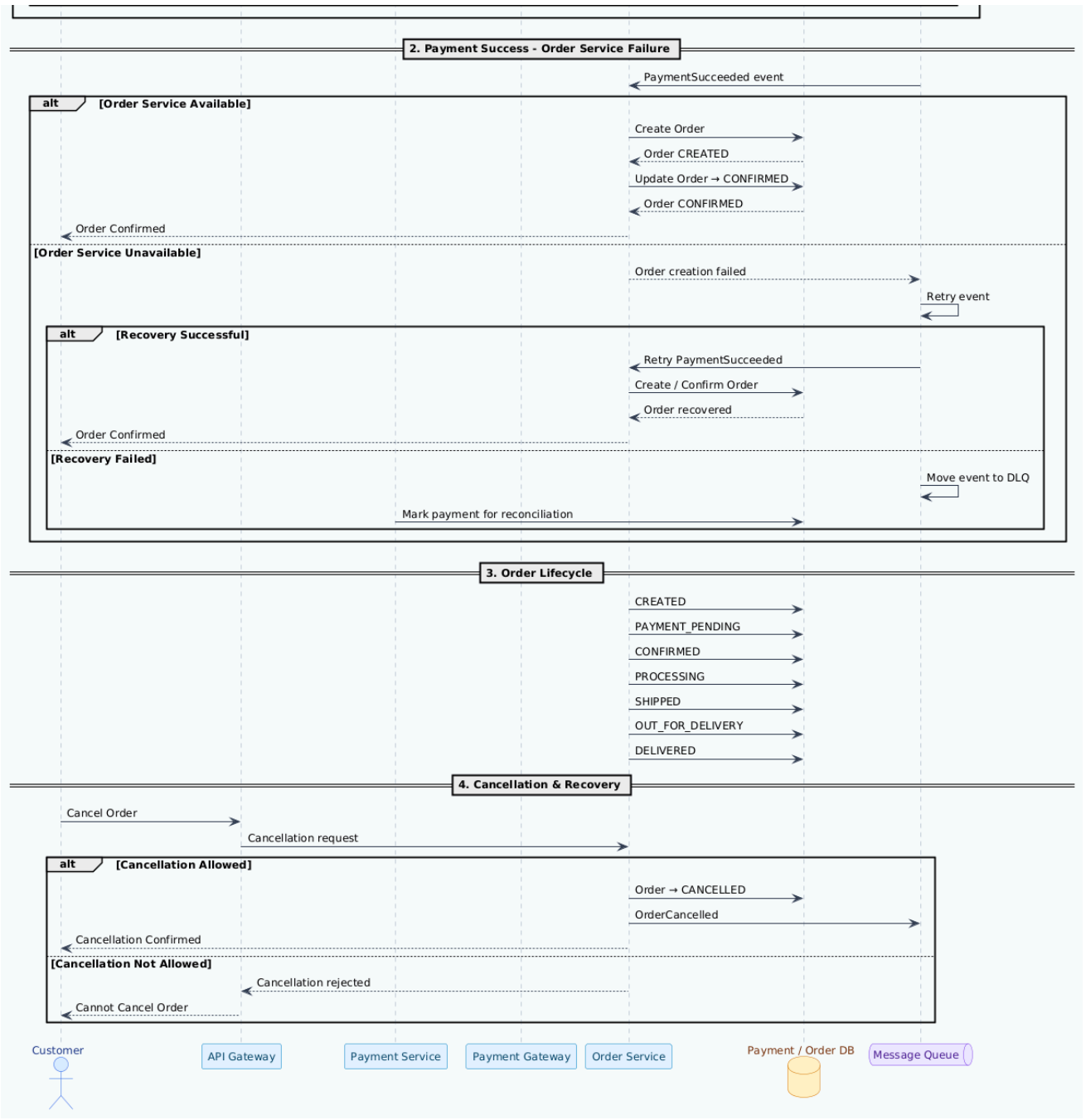
Concurrency & Inventory Design :




2. Reservation Lifecycle
Start Payment
(reservationId)
Update reservation
Update status → PAYMENT_PENDING
Payment initiated
alt
[Payment Successful]
Payment success
Update status → CONFIRMED
Reservation CONFIRMED
Order confirmed → SOLD
Reservation SOLD
Payment success
200 OK
(Order confirmed)
[Payment Failed / Timeout]
Payment failed / Timeout
Update status → RELEASED
Return reserved quantity → AVAILABLE
Inventory released
Reservation released
200 OK
(Reservation released)
3. Last Unit Overselling Prevention
Request 1 unit
(simultaneously)
Request 1 unit
(simultaneously)
Atomic update for Customer A
Atomic update for Customer B
A: 1 row updated → SUCCESS
B: 0 rows updated → OUT OF STOCK
Reservation SUCCESS
Reservation FAILED
Initial stock = 1
Final stock = 0
Overselling = 0
Customer
API Gateway
Inventory Service
Inventory DB
Outbox / Message Queue
Customer A
Customer B

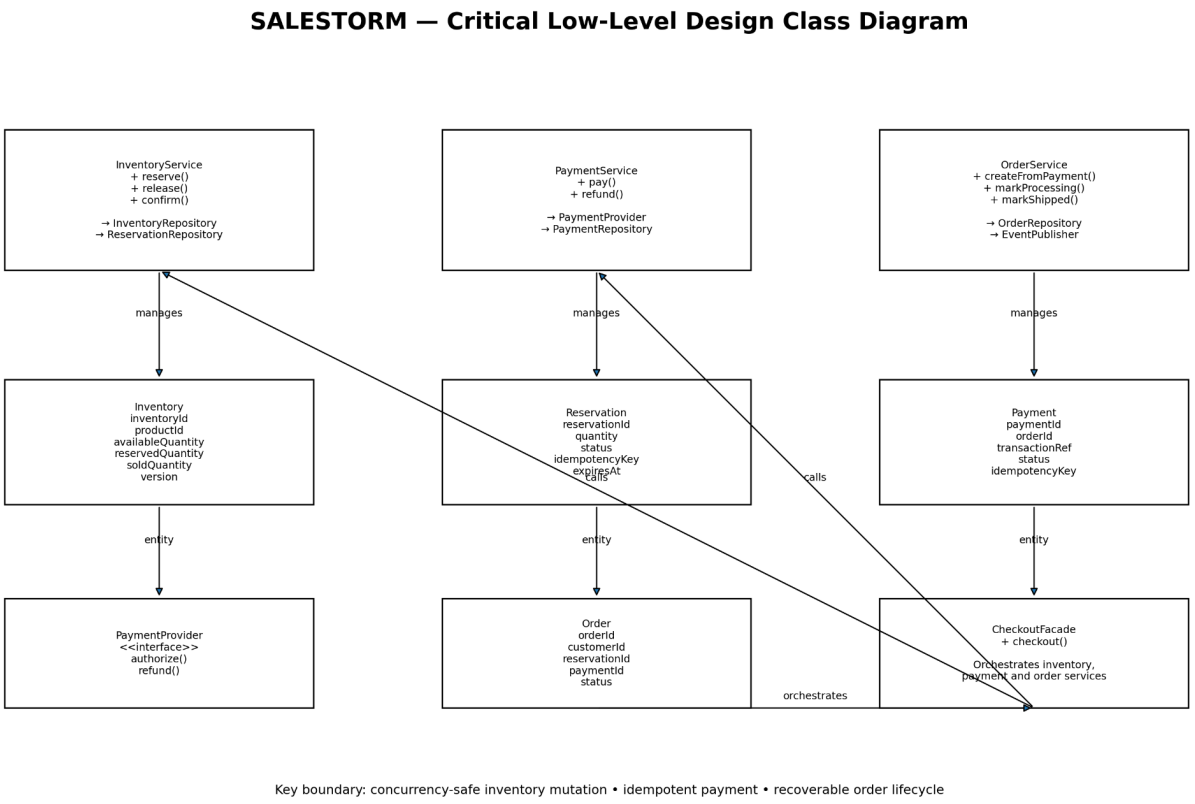
Payment & Order Design:




2. Payment Success - Order Service Failure
PaymentSucceeded event
alt
[Order Service Available]
Create Order
Order CREATED
Update Order → CONFIRMED
Order CONFIRMED
Order Confirmed
[Order Service Unavailable]
Order creation failed
Retry event
alt
[Recovery Successful]
Retry PaymentSucceeded
Create / Confirm Order
Order recovered
Order Confirmed
[Recovery Failed]
Move event to DLQ
Mark payment for reconciliation
3. Order Lifecycle
CREATED
PAYMENT_PENDING
CONFIRMED
PROCESSING
SHIPPED
OUT_FOR_DELIVERY
DELIVERED
4. Cancellation & Recovery
Cancel Order
Cancellation request
alt
[Cancellation Allowed]
Order → CANCELLED
OrderCancelled
Cancellation Confirmed
[Cancellation Not Allowed]
Cancellation rejected
Cannot Cancel Order
Customer
API Gateway
Payment Service
Payment Gateway
Order Service
Payment / Order DB
Message Queue

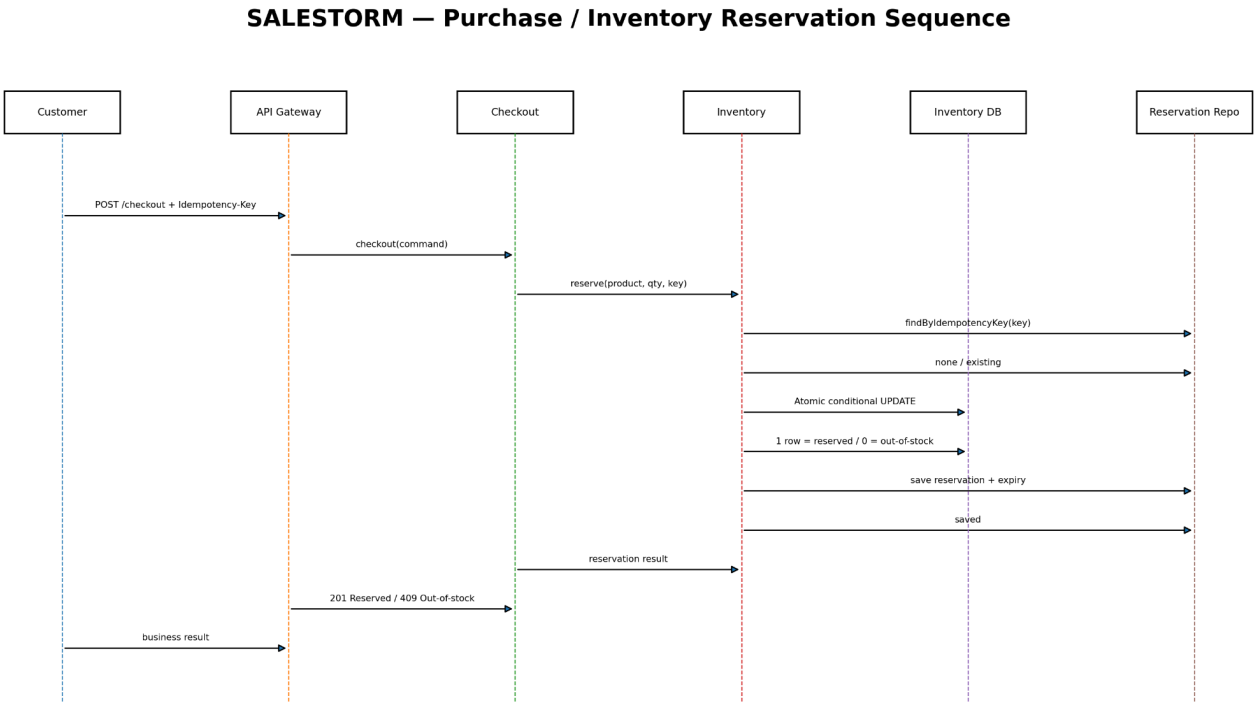
Low-Level Design:

Class Diagram:

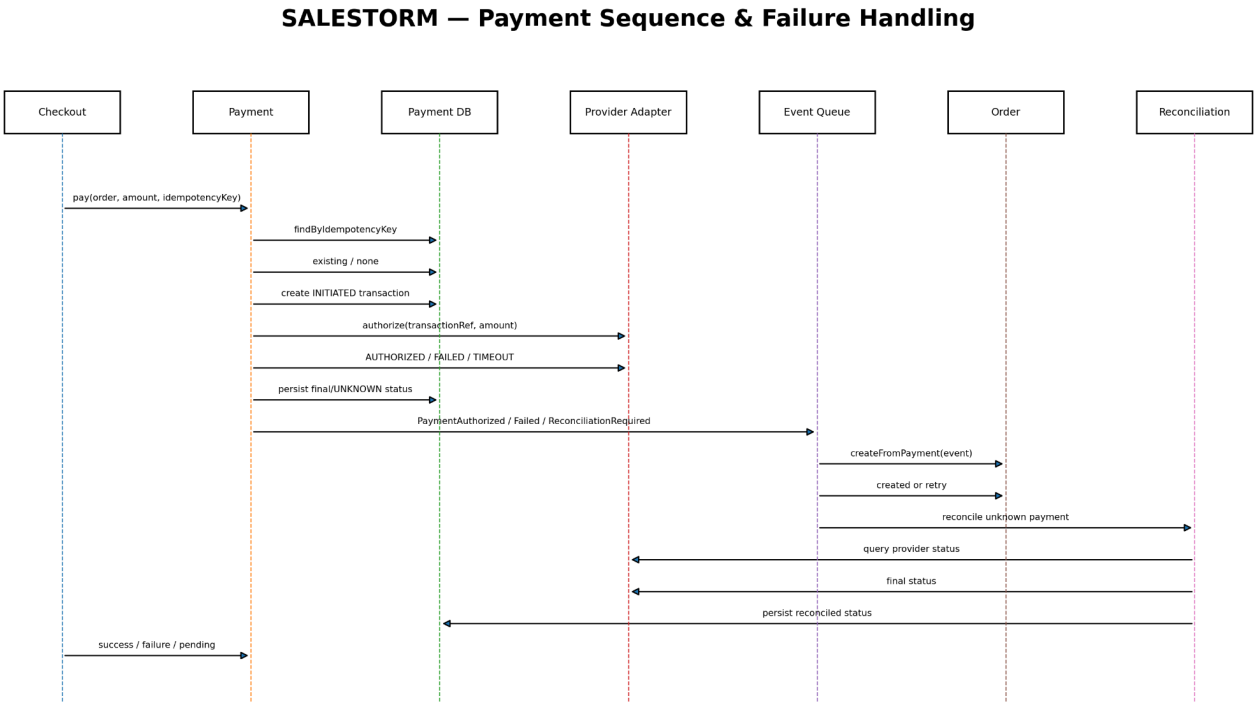


Key boundary: concurrency-safe inventory mutation • idempotent payment • recoverable order lifecycle

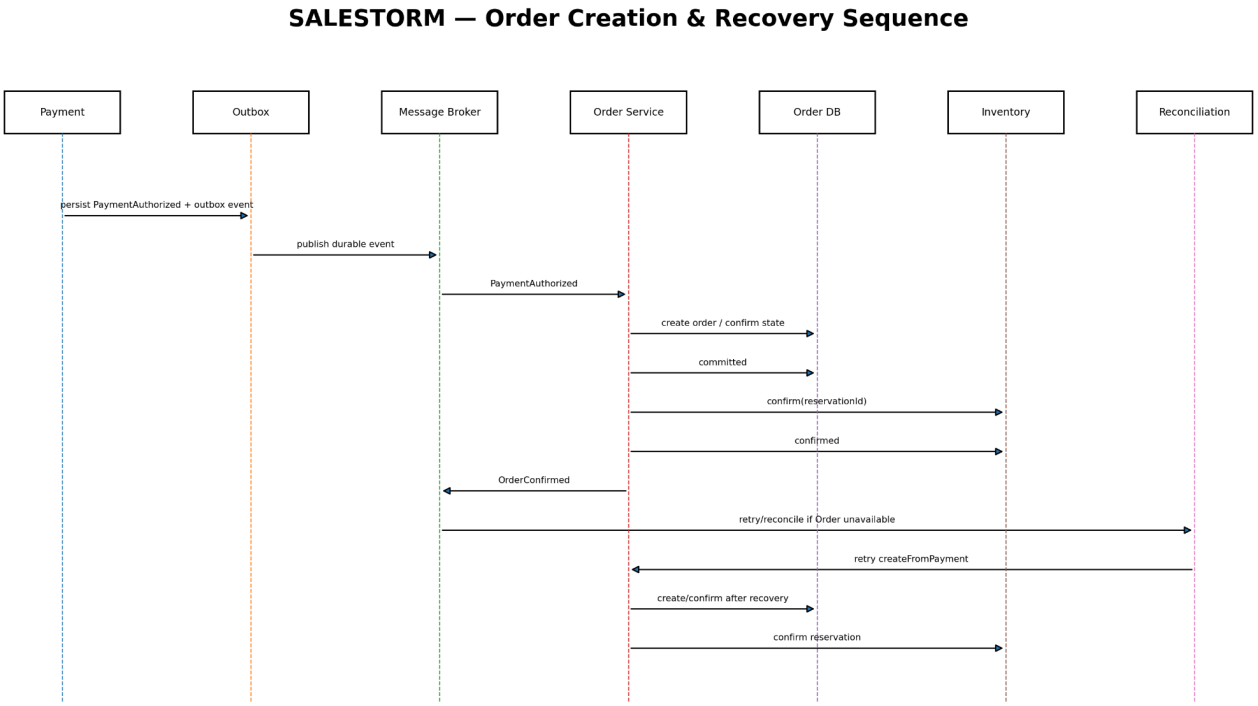
Purchase Reservation Sequence:



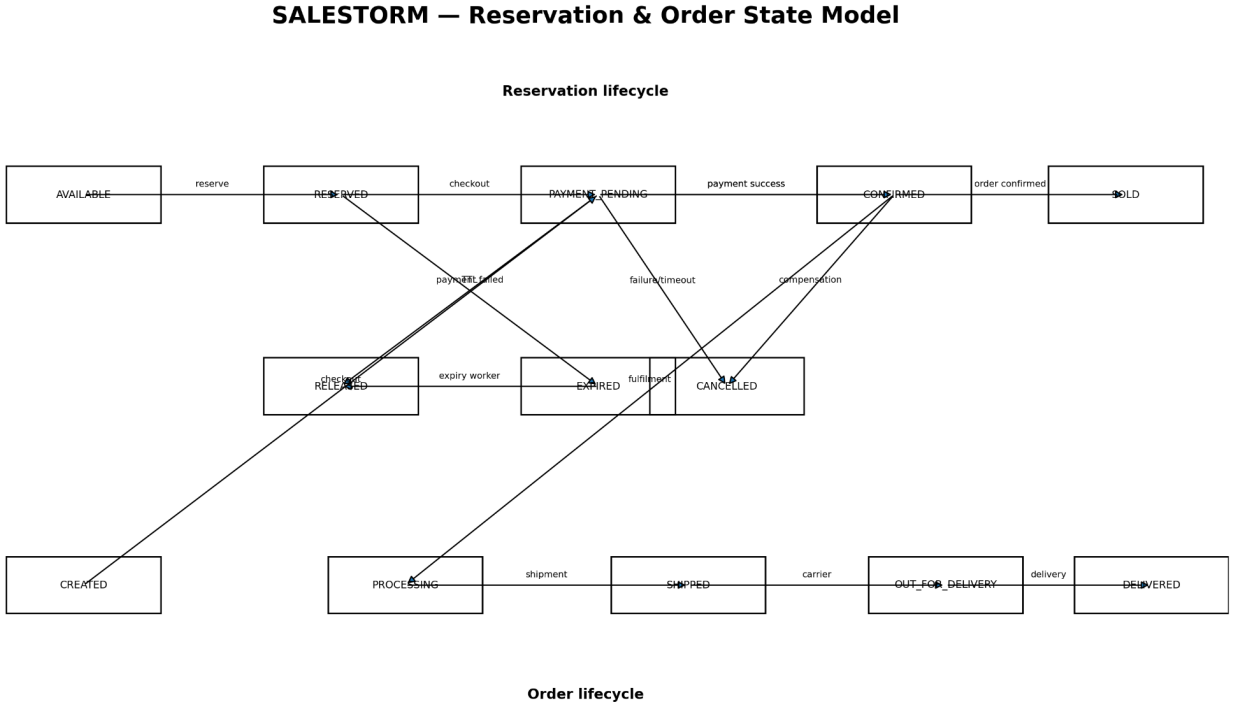
Payment Sequence:



Order Sequence:



State Diagram:



SOLID & Design Patterns:

1. Single Responsibility Principle (SRP)

- `InventoryService` owns inventory reservation/release/confirmation rules.
- `PaymentService` owns payment orchestration and payment state.
- `OrderService` owns order lifecycle transitions.
- `NotificationService` (event consumer) owns customer notification delivery. Why: changes to payment logic do not require changing inventory or order logic.

2. Open/Closed Principle (OCP)

- `PaymentProvider` is an interface.
- `PaymentService` accepts a `PaymentProvider` abstraction.
- New providers such as `ProviderA`, `ProviderB`, or `ProviderC` can be added without changing core checkout logic.
- Pricing can similarly use a `PricingStrategy` interface. Why: new business variants extend behavior rather than modifying stable core logic.

3. Liskov Substitution Principle (LSP)

- Every concrete `PaymentProvider` must honor the same contract: `authorize()` returns a valid `PaymentResult` and `refund()` returns a valid `RefundResult`.
- Checkout must work correctly regardless of which provider implementation is injected. Why: a provider implementation must not require special-case behavior from its caller.

4. Interface Segregation Principle (ISP)

Use focused interfaces rather than one large interface:

- `PaymentProvider`
- `InventoryRepository`
- `ReservationRepository`
- `OrderRepository`
- `EventPublisher` Why: each class depends only on capabilities it actually needs.

5. Dependency Inversion Principle (DIP)

High-level services depend on abstractions:

- `CheckoutFacade -> InventoryService / PaymentService / OrderService`
- `PaymentService -> PaymentProvider`
- `InventoryService -> InventoryRepository / ReservationRepository`
- `OrderService -> OrderRepository / EventPublisher` Why: infrastructure providers can change without rewriting business rules.

Recommended dependency rule

Domain/application code must not directly instantiate external payment SDKs or database clients. Factories, adapters and repositories isolate those details.

Design Pattern Mapping:

SALESTORM — Design Pattern Mapping

Pattern	SALESTORM Application	Problem Solved	Trade-off
Strategy	Pricing strategy / payment selection / delivery strategy	Select behavior at runtime	More interfaces/classes
Factory	`PaymentProviderFactory`	Centralizes provider creation	Factory becomes another abstraction to maintain
State	Order and reservation lifecycle	Prevents invalid state transitions	More state objects/logic
Observer	Order/payment events and notifications	Decouples event producers from consumers	Eventual consistency and debugging complexity
Adapter	External payment/delivery providers	Converts external APIs to internal interfaces	Extra wrapper layer
Repository	Inventory/Reservation/Payment/Order persistence	Isolates database details	Additional abstraction and mapping
Facade	`CheckoutFacade`	Provides one entry point for checkout orchestration	Facade can become too large if responsibilities leak into it
Circuit Breaker	External payment/delivery calls	Stops repeated calls to failing dependencies	Adds state, monitoring and recovery complexity

Pattern 1 — Strategy

`PricingStrategy` or `DeliveryStrategy` can be selected without changing checkout. Example implementations: `FlashSalePricingStrategy`, `StandardPricingStrategy`.

Pattern 2 — Factory

`PaymentProviderFactory.create(providerName)` returns the configured provider implementation.

Pattern 3 — State

Reservation states: `AVAILABLE -> RESERVED -> PAYMENT_PENDING -> CONFIRMED -> SOLD`, with failure paths to `RELEASED/EXPIRED`. Order states follow the required lifecycle from the brief.

Pattern 4 — Observer

`PaymentAuthorized`, `OrderConfirmed`, `OrderShipped` events are consumed by notification, analytics and fulfilment components independently.

Pattern 5 — Adapter

A provider-specific SDK is wrapped by an adapter implementing `PaymentProvider`. Core payment logic never depends on the vendor SDK.

Pattern 6 — Repository

Repositories provide persistence operations while business services remain independent of SQL/ORM details.

Pattern 7 — Facade

`CheckoutFacade` orchestrates reservation, payment and order creation through service interfaces.

Pattern 8 — Circuit Breaker

For unstable external payment/delivery dependencies: Closed -> Open after threshold -> Half-Open after recovery interval -> Closed on successful probes.

API & Event Design :

1. API Design Principles

The API design exposes the business workflow required by SALESTORM while keeping immediate customer-facing decisions synchronous and using asynchronous business events/messages for downstream processing and recovery. The API contracts below are limited to the SALESTORM workflow and do not introduce unrelated services.

- Authentication: protected customer and operational APIs require authenticated access.
- Authorization: callers may perform only operations permitted for their role/resource.
- Idempotency: business-changing operations that may be retried use an Idempotency-Key; duplicate requests return the existing business outcome rather than creating a second transaction.
- Error handling: APIs use a consistent error structure with an HTTP status, error code, message, correlation identifier and retryability indication.
- Synchronous communication: used where the caller needs an immediate business decision, such as product reads, cart operations and reservation acceptance/rejection.
- Asynchronous communication: used for durable downstream propagation, retries and recovery such as payment-success-to-order processing and order/shipment/notification events.

2. Customer / Product APIs

Method	Endpoint	Purpose	Auth	Request	Response	Errors
GET	/api/v1/products	Product discovery/listing	Customer auth as	Query: category, search, page,	200: product list	400 invalid query; 401
			required	limit		unauthorized; 500 internal f
GET	/api/v1/products/{productId}	Product details	Authenticated	Path: productId	200: product details	400 invalid ID; 404 product
			customer			unavailable; 500 internal fai
GET	/api/v1/products/{productId}/avail	Product availability view	Authenticated	Path: productId	200: availability view	400 invalid ID; 404 product
	ability		customer			unavailable; 503 service
						unavailable

3. Cart APIs

Method	Endpoint	Purpose	Auth	Request	Response	Errors
POST	/api/v1/carts	Create cart	Required	Customer context; optional	201: cart	400 validation; 401; 500
				initial items		
PATCH	/api/v1/carts/{cartId}	Update cart	Required; owner	Path cartId; cart update body 200: updated cart		400 validation; 401; 403; 40
			only			409 conflict
POST	/api/v1/carts/{cartId}/items	Add item	Required; owner	productId, quantity	201: cart with item	400 validation; 401; 403; 40
			only			409 unavailable
DELETE	/api/v1/carts/{cartId}/items/{itemId}	Remove item	Required; owner	Path cartId/itemId	204: removed	401; 403; 404
			only			
GET	/api/v1/carts/{cartId}	View cart	Required; owner	Path cartId	200: cart	401; 403; 404; 500
			only			

4. Flash-Sale Purchase APIs

4.1 Buy / Purchase

The purchase API is the customer-facing entry point for the critical flash-sale scenario. It must not imply a read-then-unprotected-write inventory operation. The request is accepted only when the inventory/reservation operation can establish a valid reservation under the approved concurrency boundary.

POST /api/v1/purchases

Authentication: required customer authentication.

Headers: Authorization; required Idempotency-Key; recommended X-Correlation-Id.

Request: cartId, productId, quantity.

Success: reservationId, purchaseReference, status=RESERVED, expiresAt.

201 Created: purchase/reservation accepted.

409 Conflict: INVENTORY_UNAVAILABLE or RESERVATION_FAILED.

400: invalid purchase request.

401/403: authentication/authorization failure.

409 duplicate: if the same Idempotency-Key has already produced an outcome, return the stored outcome rather than create another reservation.

Illustrative request:

```
{"cartId":"cart_123","productId":"prod_100","quantity":1}
```

Illustrative successful response:

{"purchaseReference":"pur_123","reservationId":"res_123","status":"RESERVED","expiresAt":"2026-10-05T12:30:00Z"}

The API is designed for the official scenario of 10,000 concurrent purchase requests competing for 100 units. Requests that cannot establish a valid reservation receive an appropriate inventory/reservation failure response; they do not create successful sales.

5. Reservation APIs

Method	Endpoint	Purpose	Auth	Request	Response	Errors
POST	/api/v1/reservations	Create reservation	Required	productId, quantity, cartId;	201 RESERVED	400; 401/403; 4 INVENT
				Idempotency-Key required		UNAVAILABLE/
						FAILED; 503
GET	/api/v1/reservations/{reservationId}	Get reservation	Required	Path reservationId	200 status + expiry	401/403; 404
POST	/api/v1/reservations/{reservationId}/confi	Confirm reservation	Required	Path reservationId;	200 CONFIRMED	401/403; 404; 4 expired
	rm			Idempotency-Key		state
POST	/api/v1/reservations/{reservationId}/rele	Release/cancel	Required	Path reservationId;	200 RELEASED	401/403; 404; 4 invalid/a

6. Checkout API

POST /api/v1/checkout — validates the cart and associated reservation state and starts the checkout/payment workflow. Authentication is required. The request contains cartId and reservationId. Idempotency-Key is required because repeated checkout requests must not create duplicate downstream business transactions.

201/202: checkout accepted with checkoutReference, reservationId and payment status. 400: validation failure. 401/403: access failure. 409: reservation missing, expired, unavailable or invalid state. 503: temporary dependency failure.

7. Payment APIs

Endpoint	Purpose	Auth	Request	Response	Errors
/api/v1/payments	Initiate payment	Required	reservationId, orderReference,	202 PAYMENT_PENDING +	400; 401/403; 409 invalid
			amount, currency;	paymentId +	gateway failure; 504 unc
			Idempotency-Key required	transactionReference	timeout
/api/v1/payments/{paymentId}	Get payment status	Required	Path paymentId	200 status +	401/403; 404; 503
				transactionReference	
/api/v1/payments/{paymentId}/confirm	Confirm/reconcile	Required	Path paymentId; Idempotency-Key 200 payment result		401/403; 404; 409 invalid

8. Order APIs

Endpoint Purpose Auth Request Response Errors /api/v1/orders Create order after valid Service-authenticated/ paymentId, reservationId, 201 orderId + status 400; 409 duplicate/inva payment/reservation internal workflow customerId, items; Idempotency-Key 503 temporary unavail required /api/v1/orders/{orderId} Retrieve order Required; Path orderId 200 order details/status 401/403; 404; 500 owner/authorized role /api/v1/orders/{orderId}/cancel Cancel where business Required; authorized Path orderId; Idempotency-Key 200 cancelled/updated 401/403; 404; 409 inva
state permits caller order

Order lifecycle:

CREATED PAYMENT_PENDING CONFIRMED PROCESSING SHIPPED OUT_FOR_DELIVERY DELIVERED

Invalid operations are rejected when they would violate the lifecycle; for example, an order cannot be moved directly to a later state without the required preceding business transition, and cancellation is permitted only where the business state allows it.

9. Shipment / Tracking APIs

Method	Endpoint	Purpose	Auth	Request	Response	Errors
GET	/api/v1/orders/{orderId}/shipment	Get shipment status	Required	Path orderId	200 shipment status	401/403; 404; 503
GET	/api/v1/shipments/{shipmentId}/tracking	Get delivery tracking	Required	Path shipmentId	200 tracking/status	401/403; 404; 503

Only shipment status and delivery-tracking operations required by the SALESTORM workflow are defined here.

10. API Error Model

All error responses use a consistent structure:

{"code":"INVENTORY_UNAVAILABLE","message":"Requested quantity is not available.","correlationId":"corr_123","retryable":false}

HTTP	Code	Meaning	Safe client action
400	INVALID_REQUEST	Request is malformed or fails validation.	Correct the request; do not blindly retry.
401	UNAUTHORIZED	Authentication is missing or invalid.	Authenticate and retry if appropriate.
403	FORBIDDEN	Authenticated caller is not authorized.	Do not retry without authorization change.
404	RESOURCE_NOT_FOUND Requested product, reservation, payment, order or shipment		Verify the resource identifier.
		does not exist.	
409	PRODUCT_UNAVAILABLE Requested product cannot currently be purchased.		Refresh product state / stop purchase attempt.
409	INVENTORY_UNAVAILABL	Requested inventory cannot be reserved.	Do not assume stock; inform caller and stop.
	E		
409	RESERVATION_FAILED	Reservation could not be established.	Do not create payment; retry only according to
			workflow policy.
409	RESERVATION_EXPIRED Reservation is no longer valid.		Create a new valid reservation if permitted.

17. Synchronous vs Asynchronous API/Event Map

	Type	Producer		Consumer	Reason
discovery/details/availability	Synchronous	Customer		Product read	Immediate user-facing result
create/update/item/view	Synchronous	Customer	Cart	Cart operation	Immediate cart response
+ reservation decision	Synchronous	Customer	Purchase/Inventory	Purchase request	Immediate reservation success/failure is
		workflow			required
status/confirm/release	Synchronous API	Customer/workflow		Reservation lifecycle operation	Immediate state result
		Reservation			
initiation	Synchronous	Customer	Checkout	Validate reservation and begin payment	Immediate validation/acceptance
				workflow	
initiation/status	Synchronous API	Checkout/Customer		Payment Payment command/status	Immediate acceptance/status
/ PaymentFailed	Asynchronous	Payment	Order/Inventory	Business event	Durable downstream propagation and
		workflow			recovery
after payment success	Asynchronous-capable	Payment workflow	Order	Order creation	Survives temporary Order Service failure
		Service			
lifecycle events	Asynchronous	Order	downstream	Fulfilment/notification	Decoupling and retry
		consumers			
reads	Synchronous	Customer	Shipment/Tracking Status retrieval		Immediate status view

18. API / Event Traceability

Business requirement	API / Event	Responsible service
Product discovery	GET /api/v1/products	Product Service
Product details	GET /api/v1/products/{productId}	Product Service
Product availability	GET /api/v1/products/{productId}/availability	Product/Inventory view
Cart	/api/v1/carts*	Cart Service
Purchase	POST /api/v1/purchases	Purchase/Checkout workflow
Inventory reservation	POST /api/v1/reservations	Inventory/Reservation Service
Reservation expiry/release	ReservationExpired / ReservationReleased	Inventory/Reservation Service
Payment	POST /api/v1/payments	Payment Service
Payment recovery	Payment status/confirm + PaymentSucceeded/Failed	Payment Service
Order	POST /api/v1/orders + OrderCreated/Confirmed	Order Service
Shipment	GET /api/v1/orders/{orderId}/shipment	Shipment/Order workflow
Delivery tracking	GET /api/v1/shipments/{shipmentId}/tracking	Tracking/Shipment workflow
Duplicate request protection	Idempotency-Key on business-changing APIs	Owning service
Payment-success / Order failure recovery	PaymentSucceeded CreateOrder async message	Payment Service Order Service

Stage 8 — Architecture Decisions & Trade-offs

Baseline note: This ADR uses the SALESTORM brief and the previously established Stage 1–7 workflow/API baseline available for this submission. The official brief requires architecture decisions to be recorded as ADRs and specifically calls for comparison of SQL vs. NoSQL, synchronous vs. asynchronous processing, optimistic vs. pessimistic concurrency and cache strategies. The brief's reference architecture includes CDN/WAF, Load Balancer, API Gateway, Product/Cart/Sale, Inventory/Reservation, Checkout, Payment, Order, Shipment/Notification, with supporting cache, message queue, database and observability infrastructure. Alternatives are allowed when justified. ADR-001 — Database: SQL vs NoSQL Title: Use a relational SQL database as the transactional system of record for core SALESTORM business state. Context: SALESTORM has inventory reservations, orders and payments with relationships and business state that must remain correct under high concurrency. The brief requires data consistency, transaction boundaries and prevention of overselling. Problem: The database must support correct inventory updates, reservations, order state and payment references while also supporting scale.

Options Considered

OptionSALESTORM evaluation A — SQLStrong transactions, constraints, relationships and integrity fit inventory reservations, orders and payment references. Concurrent inventory writes remain a controlled contention point. Horizontal write scaling is more complex. B — NoSQLCan provide flexible schemas and horizontal scale, but would require more application-level handling for relational integrity and transactional business state. It does not inherently remove the need for a correctness boundary around inventory. Decision: Select SQL for the authoritative transactional business state. Do not use a cache or non-authoritative store as the source of truth for inventory. Why selected: SALESTORM's highest-risk correctness requirements are inventory, reservation, payment and order integrity. Relational transactions and constraints provide a clear consistency boundary for these operations. Advantages: transactional integrity; explicit relationships; constraints; clear business-state persistence; strong fit

for inventory/order/payment records. Disadvantages / Trade-offs: write hotspots require capacity planning; horizontal write scaling is more difficult than stateless application scaling; operational HA/backup/failover are required. Impact on SALESTORM: application tiers can scale horizontally while the authoritative transactional write path remains controlled. Read-heavy product access can be scaled separately without weakening inventory correctness. Consequences: database capacity and write contention become important scaling considerations. The system must not claim that increasing application instances alone removes the transactional inventory bottleneck. ADR-002 — Communication Model: Synchronous vs Asynchronous Title: Use a hybrid communication model: synchronous for immediate customer/business decisions and asynchronous messages/events for durable downstream workflows. Context: Customers need immediate outcomes for reads, cart actions, purchase/reservation decisions and payment initiation, while order, fulfilment, notification and recovery workflows must survive temporary downstream failures. Problem: A fully synchronous chain would propagate downstream failures to the customer path; a fully asynchronous front door would make immediate inventory decisions harder to communicate.

Options Considered

OptionSALESTORM evaluation A — Fully synchronousSimple request flow, but tightly couples services. Payment/order/shipment failures can propagate back into the purchase path and reduce resilience. B — Fully asynchronousStrong decoupling and buffering, but immediate purchase/reservation decisions become harder to expose directly to the customer. C — HybridKeeps immediate decisions synchronous and uses durable events/messages for downstream propagation, retries and recovery. Decision: Select the hybrid approach already reflected in the API/event workflow. Why selected: Reservation availability, checkout acceptance and payment initiation require a clear immediate result. Payment success, order creation recovery, fulfilment, shipment and notification can be propagated asynchronously so temporary downstream failures do not invalidate an already successful upstream business transaction. Synchronous interactions: product discovery/details/availability; cart operations; purchase/reservation decision; reservation status and commands; checkout initiation; payment initiation/status; shipment/tracking reads.

Asynchronous interactions: PaymentSucceeded/PaymentFailed; payment-to-order propagation; reservation expiry/release recovery; order lifecycle events; fulfilment/shipment/notification propagation. Advantages: lower coupling; failure isolation; burst absorption; independently scalable consumers; durable recovery path. Disadvantages / Trade-offs: eventual consistency across services; duplicate message handling; additional operational complexity. Impact on SALESTORM: If Order Service is unavailable for 30 seconds after payment success, the payment result remains durable and the order-creation workflow can continue after recovery instead of failing the purchase as a whole. Consequences: every asynchronous consumer must tolerate duplicate delivery and transient failures; business state must not depend on an unacknowledged synchronous call to every downstream service. ADR-003 — Concurrency Control: Optimistic vs Pessimistic Title: Use an atomic conditional inventory reservation boundary (pessimistic-style contention control) for the critical inventory write. Context: The official scenario is 10,000 simultaneous purchase requests competing for 100 available units. The system must guarantee that successful sales never exceed available inventory and must handle the last-unit race correctly. Problem: A simple read-then-update flow can allow multiple concurrent buyers to observe the same remaining quantity unless the inventory write is protected by a consistency boundary.

Options Considered

OptionSALESTORM evaluation A — Optimistic concurrencyAllows concurrent attempts and detects version conflicts, but a flash-sale hotspot can produce many conflicting retries. Under extreme contention, retry amplification can increase database load. B — Pessimistic / atomic conditional update Places the correctness boundary on the inventory mutation so the reservation succeeds only when sufficient inventory remains. Losers fail safely instead of repeatedly retrying a stale quantity. Decision: Use the atomic conditional reservation approach established for the SALESTORM inventory design: the inventory mutation succeeds only when the required available quantity is still present. Why selected: It directly protects the strict inventory invariant. With 100 units and 10,000 competing requests, at most 100 successful reservation outcomes can consume the available quantity. If two customers compete for the final unit, only

one conditional reservation can succeed. Advantages: direct overselling protection; clear transaction boundary; deterministic last-unit behaviour; failed contenders can receive inventory-unavailable rather than repeatedly retrying stale state. Disadvantages / Trade-offs: the inventory write path is a hotspot; database capacity and transaction contention must be controlled; it does not eliminate the need for horizontal scaling around the database boundary. Impact on SALESTORM: horizontal scaling applies to stateless request handling, while inventory correctness remains protected at the authoritative write boundary. Duplicate reservation requests are separately protected by idempotency. Consequences: the design must not move the authoritative inventory counter into an eventually consistent cache. When the authoritative data store is unavailable, the safe behaviour is to fail the reservation rather than guess available stock. ADR-004 — Cache Strategy Title: Use caching for read-heavy product/catalog views, not as the authoritative inventory source. Context: The official architecture includes a cache and requires reasoning about traffic up to 500,000 requests/sec. Product discovery and product details are read-heavy, while inventory mutation is correctness-sensitive. Problem: The system needs read scalability without allowing stale cache data to cause inventory overselling.

Options Considered

StrategySALESTORM evaluation No cacheSimplest consistency model, but places all read pressure on the authoritative data layer during flash-sale traffic. Cache product/catalog readsReduces read pressure and improves read scalability. Staleness is acceptable for non-authoritative product views when the purchase path revalidates against authoritative inventory. Cache authoritative inventory counterCan reduce direct database pressure, but stale/incorrect values can create overselling unless a separately justified atomic inventory design is introduced. Decision: Cache read-heavy product/catalog information and availability views where appropriate; keep authoritative inventory reservation state outside the cache. Why selected: SALESTORM's 500,000-request/sec design envelope makes read amplification a scalability concern, but inventory correctness is a strict business requirement. A stale product view must never be allowed to authorize a sale by itself. Advantages: lower read load; improved product-discovery scalability; faster read responses; isolates read spikes from the transactional write path. Disadvantages / Trade-offs: stale reads; cache invalidation;

cache availability is another dependency; cache failure requires graceful fallback to the authoritative source where appropriate.

SALESTORM | SYSCRAFTERS 2026 — Stage 8 Architecture Decision RecordPage 3 Impact on SALESTORM: flash-sale product reads can scale horizontally while the reservation decision remains authoritative. Cache failure must degrade availability of the cached read path rather than corrupt inventory state. Consequences: cache invalidation/update behaviour must be defined for product availability views, and the purchase flow must not treat cached availability as proof that inventory can be reserved. ADR-005 — Payment Reliability and Idempotency Title: Use idempotent payment requests with a unique transaction reference and reconciliation for uncertain outcomes. Context: The SALESTORM payment workflow must handle success, failure, timeout and payment-gateway failure and must prevent duplicate payments. Stage 7 defines Idempotency-Key use and a unique payment transaction reference. Problem: A payment timeout does not prove that the gateway did not process the transaction. Blindly issuing a new payment can charge the customer twice.

Options Considered

OptionSALESTORM evaluation Blind retry as a new paymentSimple, but unsafe when the first gateway request succeeded and only its response was lost. No retry/recoveryAvoids duplicate charges but can leave legitimate payments and reservations unresolved. Idempotent payment + reconciliationReuses the same logical transaction identity and resolves uncertain results without creating a second charge. Decision: Use Idempotency-Key on payment creation and a unique transaction reference for the payment business transaction. Timeout/uncertain outcomes are reconciled rather than blindly duplicated. Why selected: This directly satisfies SALESTORM's duplicate-payment requirement while allowing recovery from gateway timeouts and failures. Advantages: duplicate-payment protection; safe retry semantics; recoverable gateway uncertainty; clear correlation between payment, reservation and order workflows. Disadvantages / Trade-offs: reconciliation adds workflow complexity and can delay order completion while payment state is uncertain. Impact on SALESTORM: payment success can safely drive asynchronous order creation, while payment failure triggers the reservation release path. Consequences: payment

consumers and recovery processing must be idempotent; timeout must not be interpreted as payment failure without checking/reconciling the transaction state. ADR-006 — Durable Asynchronous Recovery for Order Processing Title: Preserve successful payment outcomes through durable asynchronous order processing and idempotent consumption. Context: The official practical scenario explicitly includes payment succeeding while the Order Service is unavailable for 30 seconds. Stage 7 therefore models PaymentSucceeded as an asynchronous business event that can drive order creation. Problem: A synchronous dependency on Order Service would make a temporary 30-second outage appear to the customer as a failed purchase or could cause unsafe retry behaviour.

Options Considered

OptionSALESTORM evaluation Synchronous payment → order call onlySimple, but couples payment success to Order Service availability and makes downstream outage recovery harder. Durable asynchronous PaymentSucceeded → Order creation Separates payment completion from temporary order-service availability and supports retry/recovery. Drop the event on failureNot acceptable because it can lose a successful payment/order transition. Decision: Use the durable asynchronous PaymentSucceeded workflow already represented in the Stage 7 event design. Order creation is idempotent. Why selected: It directly addresses the required 30-second Order Service outage scenario while preserving the successful payment result. Advantages: failure isolation; retry; horizontal consumer scaling; no need to re-charge the customer after Order Service recovery. Disadvantages / Trade-offs: eventual consistency between payment and order state; requires durable message handling and idempotent Order Service consumption. Impact on SALESTORM: payment success remains authoritative even when Order Service is temporarily unavailable; the order can be created after recovery without duplication. Consequences: the system must monitor failed/retried messages and ensure consumers can safely process duplicates.

Trade-off Summary

SALESTORM | SYSCRAFTERS 2026 — Stage 8 Architecture Decision RecordPage 4 DecisionSelected OptionAlternativePrimary BenefitPrimary Trade-offSALESTORM

Reason DatabaseSQLNoSQLTransactional integrity for inventory/order/payment state More complex horizontal write scaling SALESTORM needs strong business-state integrity and relationships. Communication modelHybrid synchronous + asynchronous Fully synchronous / fully asynchronous Immediate decisions plus failure isolation Eventual consistency and message handling Protects customer purchase flow while supporting downstream recovery. Concurrency controlAtomic conditional / pessimistic-style inventory boundary Optimistic concurrencyDirect overselling prevention under hotspot contention Controlled write contention10,000 requests competing for 100 units requires a strong inventory correctness boundary. CacheCache read-heavy product/catalog views No cache / authoritative inventory cache Read scalability under flash-sale traffic Staleness and invalidation500,000 req/s design envelope requires read scaling without weakening inventory correctness. Payment reliabilityIdempotent payment + reconciliation Blind retry / no recoveryPrevents duplicate charges while recovering uncertain gateway outcomes More workflow complexityPayment timeout must not become a second charge. Order recoveryDurable PaymentSucceeded → idempotent order creation Synchronous payment→order only Survives temporary Order Service failure Eventual consistencyRequired 30-second Order Service outage scenario. Consistency boundary: These ADRs preserve the key SALESTORM invariants: successful sales must not exceed available inventory; duplicate business requests must not create duplicate reservations/payments/orders; payment and order workflows must recover from temporary downstream failure; and scaling must not move correctness into an uncontrolled eventually consistent cache.

